

EBERHARD-KARLS-UNIVERSITY TÜBINGEN
Wilhelm-Schickard-Institute for Compute Science
Chair Cognitive Systems

Bachelor of Science

**3D reconstruction of table tennis
trajectories from single camera footage**

Mika Thormann

Supervisor: Prof. Dr. rer. nat. Andreas Zell
Wilhelm-Schickard-Institute for
Computer Science

Start Date: 01.11.2022

End Date: 13.03.2023

Abstract Using stereo vision to track three-dimensional trajectories is a common practice when researching ball sports mechanics. Multiple cameras catching different perspectives of a scene allow highly accurate tracking, but such a setup is costly and must be installed and calibrated properly. That is why this practice is mainly carried out in experimental environments with permanently installed technology. The amount of data such systems generate is limited and yields low diversity. In contrast to that, training an agent to play a game at a high level requires large amounts of diverse data. This data can be found everywhere on the internet, but most of it consists of monocular videos that are recorded from only one perspective at a time. In this thesis, we present a technique to extract trajectories of single-viewpoint table tennis videos. Our proposed technique combines traditional computer vision to estimate the camera pose with a differential evolutionary algorithm based on a physics simulation. Our algorithm aims to be robust against occlusion and works without prior knowledge of the input videos. Like this, we try to make a considerable amount of currently unused data accessible.

Contents

1. Introduction	1
2. Camera Pose Estimation	2
2.1. Preprocessing	4
2.2. Corner position estimation	5
2.3. Corner orders and PnP	7
2.4. Postprocessing	10
2.5. Evaluation	11
3. Physics Simulation	12
3.1. Flying Movement	12
3.2. Bounce Events	14
3.3. Test Data Generation	15
4. 3D position estimation	17
4.1. Using Bounces to estimate trajectory planes	17
4.2. Differential Evolution	19
4.2.1. Choosing values for Θ	22
4.2.2. Restricting validity of individuals	23
4.2.3. Determining start and end frames of trajectories	24
4.2.4. Dealing with spin	25
4.3. Method comparison	25
5. Evaluation	27
5.1. Analysing Correlations	28
5.2. General performance	33
5.3. Tests with real data	35
5.4. Concluding Thoughts	36
A. Value Table	37
B. Abbreviations	38
Bibliography	39

1. Introduction

The main motivation for our work is to generate large amounts of data that can be used for machine learning tasks. One possible use case for this kind of data is a reinforcement learning (RL) agent playing table tennis. When selecting raw videos as input for such an RL algorithm, its performance is negatively influenced by huge amounts of distracting unnecessary data. That is because only the table, ball, and rackets are of importance for the task and everything else in the scene can be seen as noise. This makes the computation not only slower but also leaves the open challenge of finding the important parts of information for the agent. If instead, only the table's pose, the trajectory of the ball, and the movement of the rackets are used as input, the agent can learn on a much more compressed version of the given data with less noise contamination.

In stereo vision systems, the distance between cameras as well as their viewing direction relative to each other and the table is known. If an object is tracked with multiple of these given cameras, the 3D position can directly be inferred through triangulation techniques. With only one camera, however, we need to find a different method to determine the camera's relative position to the scene. To do so, we make use of standardized table sizes in table tennis. The table acts as a reference for our coordinate system and is used to estimate the camera position and orientation. Tracking the ball's position on the image plane is not part of this study, as it would force us to exceed the given amount of time for this project. Other works in the field of machine learning like TTNNet[VFB20] already solved this problem and can be used to generate image plane positions for our algorithm.

One could now imagine a camera with extremely high resolution. With this camera, it would be possible to not only estimate the horizontal and vertical camera coordinates of the ball but also the depth in the camera 3D space, as the ball becomes smaller the further away it is from the camera. This information together with the camera's relative position would be sufficient to directly convert those to real-world coordinates. As we thrive to apply our algorithm to a wide range of videos on the internet, this is very unrealistic. Typical videos are of much lower resolution than required for this to work. In addition, they are often compressed and suffer from motion blur and aliasing. That is why we have to find a solution that only uses the estimated camera pose and coordinates of the ball on the camera's image plane. We propose two approaches to this problem. The first one estimates the ball's position when it bounces, building a plane orthogonal to the table, which is then used to infer all other ball positions. The second approach makes use of a physics simulation together with the concept of differential evolution. For every 2D input trajectory, a population of individual candidate 3D trajectories is simulated, and their performance on the task is evaluated.

Going through this thesis, we will first describe how we extracted the cameras pose for every frame in a video. After that, we explain the physics model we used to generate our trajectories. These topics then lead to our final work on how to estimate the ball's 3D position. In the end, we analyze the performance of our algorithm and explain how we would further proceed to enhance it.

2. Camera Pose Estimation

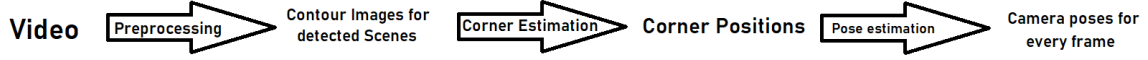


Figure 2.1.: Simple visualization of our process pipeline

Before getting to the algorithm, we would like to mention the assumptions we made for input videos. Those might sound like harsh restrictions, but we observed that most professionally recorded table tennis matches meet all four of them.

- 1) The video was recorded by stable cameras and only shows one perspective at a time.
- 2) The table is placed in the center of the video.
- 3) The color of the table is green or blue. It should also be different from the background color in the area close to the table.
- 4) If the perspective changes, there must be a cut that clearly separates one scene from the other. These cuts should be fast to reliably get spotted.

We use the pinhole camera model for our estimation, where the camera's intrinsics are defined by distortion coefficients C_{dist} and a camera matrix M_{cam} that gives the focal length of the lens. The camera's distortion coefficients are not given for the videos and are quite difficult to estimate without further information. But as they are usually quite low for normal cameras, we assume them to be zero for our modeling.

$$C_{dist} = \begin{pmatrix} 0 & 0 & 0 & 0 \end{pmatrix} \quad (2.1)$$

We estimate the camera matrix by extracting the image width W and height H and using them to estimate the focal length. This is a commonly used practice when the actual values are unknown and works with any given resolution.

$$M_{cam} = \begin{pmatrix} W & 0 & W/2 \\ 0 & H & H/2 \\ 0 & 0 & 1 \end{pmatrix} \quad (2.2)$$

For the estimation of the ball's position, some information about the position of the camera in 3D space is necessary. We choose to create such knowledge by calculating its relative position to the table. The table is consistently present in any video and always has a fixed size, making it a reliable candidate for this purpose. To extract the information it provides, we use iterative Perspective and Point (PnP). By defining points in 3D space and giving their corresponding 2D

position on the image plane, PnP directly infers the camera's pose. This pose is defined by the position and orientation of the camera in the chosen 3D space, expressed as a translation vector T and a rotation vector R .

We choose to make the center of the table the origin of our 3D space, with the x-axis going across the longer extent of the table as shown in 2.2. To make use of the PnP method, we now define points on the surface of the table that we are able to detect well in given images. We decided to use the four corners of the table for this purpose because they are an easily detectable geometrical feature. In addition, they give the outer bounds of the table, so even if there is an error present in their calculated positions, the pose is only minimally affected.



Figure 2.2.: Example of how we place our axes to represent the 3D space. The axes are scaled to match the dimensions of the table. {x=red, y=green, z=blue}

2.1. Preprocessing

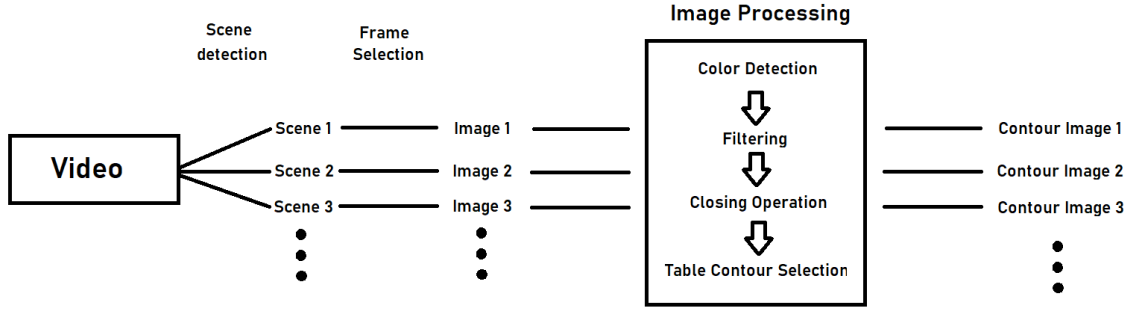


Figure 2.3.: Visualization of our preprocessing pipeline which takes a video as input and generates table contour images for every detected scene

Scene detection First, we need to divide a given video into separate scenes, each containing only one perspective. By observing changes in color and intensity, fast cuts can easily be detected and used to determine the start and end frames of scenes. A cut is triggered whenever the intensity of the change reaches a certain threshold. Our further explained algorithm will be applied separately to each detected scene.

Frame selection We later want to be able to determine the positions of the corners on the image plane. As only the table is of importance for that, our first goal is to isolate the pixels that belong to it, hence, making it easy to analyze. To accomplish this, we make use of the table's color which is either green or blue. To find out what color the table is, we rely on the assumption that it can be found in the center of the image. The prevalence of green and blue pixels in the center area of the first frame is observed and the more common color is selected. If a pixel is green or blue is decided by handcrafted color ranges C_{blue} and C_{green} that are also used for upcoming filtering of the image.

To make further processing easier, we would like to choose the frame with the least table occlusion for the scene we observe. This is done by selecting the frame with the highest amount of pixels of the same color as the table, assuming that this means that it is the frame where most of the table is visible.

Image Processing With the given color range, we are also able to filter the image, assigning white color to pixels that fall into it and black to ones that do not. We then apply a closing operation on the image, filling up possible shallow occlusion areas like those created by a net. As not only the table can fall into the color range, we still have to isolate the table from other objects of the same color. We do so by determining contours for every white area in the image and calculating their bounding boxes. Once again relying on the assumption of a centered table placement in the image, we can reduce the observed contours to those which have a box that encapsulates the center point of the image. From those we assume the one with the largest area to be the contour of the table. If there is another contour that is at least 0.25 times as large, we keep them both. This makes sense, as the net's occlusion might cut the table's area in half, leaving us with two areas that contribute to the table.

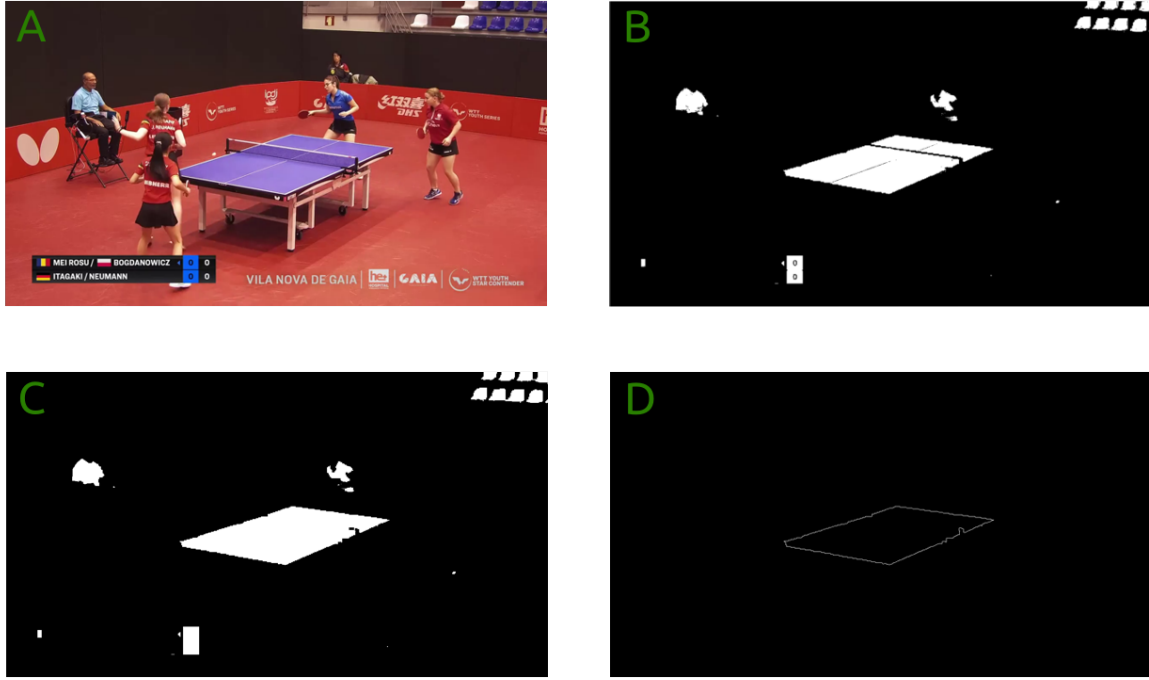


Figure 2.4.: Visualized preprocessing steps: A) Unprocessed image B) Filtered Image C) Filtered image with applied closing operation D) Table contours

2.2. Corner position estimation

The contours of the table's area could now be used to directly infer the corners' positions. This would be done by choosing those points closest to the edges of the bounding box. Unfortunately, that is only possible if we knew that there is no occlusion hiding the table. Occlusion can either directly hide corners on the image or cut off parts of the table that contain one. This is why we tried to reduce its effect as much as possible before when selecting the best frame to analyze. Unfortunately, that technique is not guaranteed to eliminate all occlusion.

To reconstruct mentioned cut-off corners, we need to rely on other features which are less affected by occlusion. One such feature that is able to directly give the position of the corners are the table's edges. For any reasonable amount of occlusion, parts of the edges are still intact, as they cover more space on the image than a corners point does. This makes it possible to detect them more reliably. We can then directly compute the intersections of those lines to give us an estimation of the corners' positions.

To approximate lines, given by the edges of the table, we decided to use the Hough Line Transformation algorithm [DH72]. This algorithm expresses lines in the Polar system, making it possible to directly infer if a line on the image is close to many pixels. As the contours should consist of the table's edges, we can directly apply the algorithm to the contour image. We iteratively use it multiple times while simultaneously decreasing the number of points needed to detect a line. This makes it possible to detect the edge lines independently from the size of the table in the image. It also accounts for possible shortening of the edges due to occlusion. We assume the table's edges to be the longest lines in our contours, making them the first to be detected. That is why we can terminate if we find more than five lines. This number allows

there to be one extra line that might be created when the table was split in half by the net. The line created in this situation always yields points that are further from the image limits than the corners. Therefore, we select those four intersection points to be our corner estimations, which are closest to the four edges of the image. At the same time, setting a line count limit also ensures that the program stops in situations where there are other lines created by occlusion. In these situations, we can not reliably determine which intersections represent the corners. Hence, further computation of more lines is unnecessary.

We also make sure that no edge will be used twice by sorting out lines that are very similar to already detected ones. The similarity is measured by calculating the distance between the beginning- and endpoints of the lines in the image. If both are below the threshold $t_{sim} = W/20$, we assume that the underlying edge was already detected and the newly discovered line will be discarded. This threshold scales with the image size, making it work on any resolution.

After all mentioned steps to ensure accurate corner positions, there are still cases in which our method fails. Those appear if heavy occlusion creates lines that are longer than the table's edges in the image. This is difficult to deal with, as such lines could create intersections anywhere on the image. In fact, we did not manage to always determine which lines are the table's edges. All of our ideas failed with our goal in mind to work on a wide variety of perspectives. Hence, we came up with a postprocessing solution to this problem that will be explained later in the text.



Figure 2.5.: Found lines on the image and their intersection points, giving corner positions.

2.3. Corner orders and PnP

For now, assuming we found the 2D positions of the table's corners $C_{i \in \mathbf{N} \leq 4}$, we can make use of the iterative PnP method to finally receive our estimation of the camera's pose. For this to work, we need the input order of the 2D positions to match the order of the 3D positions. The set of possible input orders O initially consists of all 16 possible permutations of corner orders. We did not manage to directly infer which 3D corner a detected 2D corner belongs to while operating on a wide variety of perspectives. But we were able to group them by determining which point pairs are diagonally connected over the table. This is done by analyzing the lines that built the intersections for every corner. We generate two pairs of points, where their underlying lines are pairwise distinct (2.6). Two resulting pairs of points (C_1, C_2) and (C_3, C_4) are now clear as we can choose the order of 3D positions to pair points in the same way, but we are still unable to produce a fixed order for every perspective. This is because we can still permute the orders inside of our pairs, leaving us with four possible permutations in the set of possible orders O .

$$O = \left\{ \begin{array}{l} (C_1, C_2, C_3, C_4), \\ (C_1, C_2, C_4, C_3), \\ (C_1, C_3, C_2, C_4), \\ (C_1, C_3, C_4, C_2) \end{array} \right\} \quad (2.3)$$



Figure 2.6.: Construction of the first point pair, given by two distinct sets of lines. The corner pair is marked with green crosses while the line color shows how the two sets are distinct.

We can further reduce this number of possibilities by fixing the first two corners' order to be C_1, C_2 . That way, we are only left with the two permutations (C_1, C_2, C_3, C_4) and (C_1, C_2, C_4, C_3) . Choosing the wrong order for the first pair results in the table being flipped upside down in our PnP estimation, as can be seen in 2.7. But we can detect such a mistake and fix it as follows.

For now, assuming that the second pair is in the right order in regards to the first pair, we have one fixed order for all four points. Using this order, we apply the PnP to receive a rotation vector R and translation vector T of the table. We can transform the rotation vector to the form of a rotation matrix M_R by applying the Rodrigues formula. With this rotation matrix, we can check if the estimated camera faces downwards or upwards by calculating

$$M_R * \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} > 0 \quad (2.4)$$

If this equation is true, our camera faces upwards. As we know that the camera actually faces downwards, filming the top surface of the table, this means that we need to fix the pose. We do so by applying a 180° rotation around the y-axis on the camera matrix:

$$M_R = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -1 \end{pmatrix} * M_R \quad (2.5)$$

We can simply transform the rotation matrix back to vector form, again by using the Rodrigues formula. Meanwhile, the translation vector does not have to be changed as it is given relative to the camera's viewing direction. Like this, we made sure that our camera faces the tables upside and not its downside.



Figure 2.7.: Left: Example where the camera was estimated to be facing upwards instead of downwards, caused by the first corner being chosen wrong. Right: The correct estimation, which is the same as on the left picture, but rotated 180° around the y-axis.

While we now have a solution to fix the estimated pose for a wrong choice of order in the first pair, we still need to address the unknown order of the second pair. Taking the wrong order here must result in the PnP algorithm yielding bad results. This is because swapping the order of the points also causes the length and width of the table to be swapped (2.8).

We can use this effect to indicate which second pair order is right, by running PnP on both possible permutations. The estimated pose that yields better results should then be our final choice. But to determine which pose fits the point better, we still need to evaluate how well it performed. To test the PnP output, we use the pose it provides to project our defined 3D corner positions on the 2D camera plane by calculating

$$C_i^* = M_R * C_i + T \quad (2.6)$$

where C_i^* is the projection of a corner on the image plane. A perfect PnP pose estimation would mean that those points now match the positions of the found corner positions. Hence, they should be close to each other when it worked well. The order we defined earlier tells us which points resemble the same corners (C_i, C_i^*) , being those with the same index i . We can now calculate the point distance for every pair and use it as a measurement for performance. Finally, our performance is determined by the furthest observed distance between two points of the same index D_{max} .

$$D_{max} = \max_i \|C_i - C_i^*\| \quad (2.7)$$

Doing this with both possible permutations, we choose our final pose to be the one that reaches a lower maximum distance D_{max} . If the maximum distance still exceeds a given threshold $t_D = H/15$ for the better pose, we assume that the estimation failed because of bad corner position estimations. A failed estimation can mean that any part of the preprocessing went wrong, but most likely occurs due to earlier mentioned lines generated by occlusion. The outcome of this test is remembered, so bad estimations can be labeled as such.



Figure 2.8.: Left: A case where the first pair is in the right order and the second is not. Right: Correct pose estimation.

2.4. Postprocessing

As the camera is given to be stable in all given scenes, we can safely assume that, throughout the whole video, there will be multiple occurrences of the same perspective. Hence, for every estimation that was labeled as a failure, we can copy the camera's pose of a different scene in the set of accurately estimated poses S_{acc} , if we are able to prove that the perspective is the same. We observed that when the estimation fails, usually some of the table's edges are still being detected. Therefore, a few of the actual corners should be present in found intersections. We can make use of these intersections by comparing them with the corners we found for poses in S_{acc} . If there are intersections that are close to those corners, we can safely assume that the perspective is the same in both scenes. Hence, we copy the pose of the scene, which has the largest amount of similar intersection-corner pairs. If such a pair is similar is decided by checking if their position's distance is below the threshold t_{p_dist} . The best candidate must have at least two similar corner-point pairs for the copying process to happen. Like this, cases in which only one point is randomly similar will not lead to the copying of a different perspective. You can see an example of how this works in figure 2.9. There, two similar points were found, such that the valid pose estimated for the left scene can be copied for the right scene.

It is very likely to find at least one frame without occlusion for every perspective over the course of the whole video. Hence, many scenes that the estimation failed on before now receive a correct estimation with this method. This worked much better than any preprocessing technique we tried to make the corner selection work with occlusion. It also did not find any wrong similarities between perspectives in our tests.

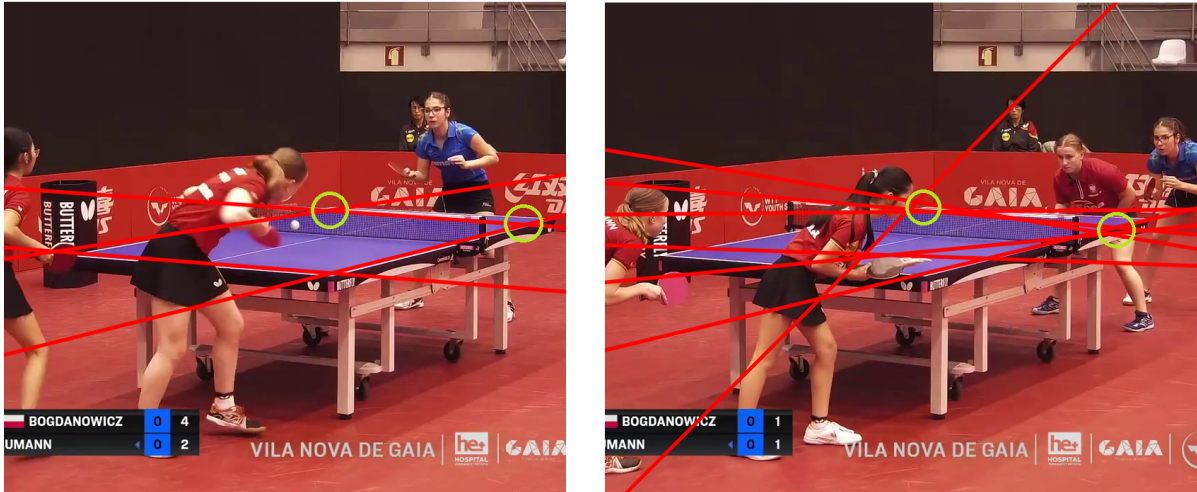


Figure 2.9.: Postprocessing example where the left picture shows the lines of a frame where the pose estimation worked and the right picture shows one where it did not work. Two similar points can be found and the pose estimated in the left frame can be copied for the right one.

2.5. Evaluation

We end up with a reliable method that is often able to indicate when it failed. It deals well with occlusion by copying perspectives in the postprocessing step. However, it is likely to fail if the angle between the camera's viewing direction and the table plane is too shallow. This happens since occlusion effects created by players and the net become stronger and more likely to happen in such a situation. We should mention here that such shallow perspectives are not very common and are exclusively used for serves.

Unfortunately, we are unable to benchmark our estimation as we could not find any data to do so. But PnP is widely being used for pose estimation and small errors in the corners position estimation should only minimally affect it. Hence, we believe that using it for our purpose is a justified choice.

3. Physics Simulation

There are two major reasons for us to construct a physics simulation of table tennis. First, we would like to be able to generate trajectories and use them for testing and benchmarking our work. Secondly, we want to implement it in our later explained evolutionary algorithm, generating 3D positions for initial ball states.

We assume that there is no airflow on the playfield. This is reasonable, as table tennis is usually played in places where there is no wind. Also, the ball must be flying or bouncing at any moment. Hence, we do not simulate rolling movement. If the ball is rolling in table tennis, one side will achieve a point and the set ends. When this happens, we are not interested in the ball's position anymore, meaning that this is a valid restriction.

For the physics involved in table tennis, there are two different scenarios we have to account for. Either that the ball is flying in the air or bouncing off the table. The bounce off of rackets is not of importance to us, as we are planning to detect and simulate single independent trajectories. To combine the two cases in one simulation, we constructed a hybrid dynamical system. In that system, flying movement is calculated with a differential equation for a given step size while bounces are treated as discrete events.

3.1. Flying Movement

For the estimation of the ball's flying movement, we have to calculate every force that acts on it in that state. These forces, together with the mass of the ball, give us an acceleration that we can apply to the ball's velocity for a given small step size. With that velocity, we can then update the ball's position. Through this, we approximate the physically accurate behavior.

Gravity and Buoyancy Gravity's acceleration is known to be $g = 9.81 \frac{m}{s^2}$. We also know the ball's mass $m = 0.0027$ kg and the force's direction to point directly downwards. Hence our equation for the gravity force is

$$F_g = m * g * \begin{pmatrix} 0 \\ 0 \\ -1 \end{pmatrix} \quad (3.1)$$

Buoyancy works directly against the gravity in direction of the z-axis. It is calculated by

$$F_b = -p * V * g * \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \quad (3.2)$$

with p being the air density and V the volume of the ball. It is a very small force, but we decided to include it, as we only have to compute it once.

Drag The drag force is assumed to be

$$F_d = \frac{1}{2} * p * S * C_D * \|v\| * v \quad (3.3)$$

where C_D is the drag coefficient and v is the airflow velocity relative to the object. This equation is widely used in other works that analyze table tennis physics. The drag coefficient for a table tennis ball is approximately $C_D = 0.4$. The relative airflow velocity relative to the object is equal to the ball's velocity, as we restricted our field of study to situations without external airflow.

Magnus Force Magnus force appears when the ball is spinning while flying. Different equations are proposed across multiple papers to calculate this force. For our simulation, we decided to use

$$F_m = \frac{4}{3} * C_M * p * \pi * r^3 * \vec{\omega} \times \vec{v} \quad (3.4)$$

where $C_M = 0.069$ is the Magnus coefficient, ω is the angular velocity, and r is the radius of the ball. This equation was proposed in [NNH10] and seems to generate reasonable trajectories in our simulation.

Final force and state update After calculating all these forces, we can sum their effects to receive the actual force working on the ball F^+ .

$$F^+ = F_g + F_b + F_d + F_m \quad (3.5)$$

At time t , the state of the ball is called S_t and contains the position p_t , velocity v_t , and angular velocity w_t . The difference in time between two consecutive states S_t and S_{t+1} is named dt . The smaller dt is, the closer the simulation becomes to real-world physics. This is because we sample the actual force values more densely along the time axis. We found that a dt of 10ms is sufficiently accurate for this project.

Initially, a starting state S_0 for our trajectory is given. For every step in our simulation, we update the position P_t at time t with the current speed over the time dt .

$$P_{t+1} = P_t + v_t * dt \quad (3.6)$$

For the update of velocity, we calculate it with the given acceleration produced by our calculated force F^+ :

$$v_{t+1} = v_t + \frac{F^+}{m} * dt \quad (3.7)$$

The angular velocity changes only minimally while the ball is flying and barely affects the trajectory. Hence, we decided to ignore it and set $\omega_{t+1} = \omega_t$.

3.2. Bounce Events

To simulate the change of the ball's state when it bounces, we decided to treat it as a discrete event that is evaluated in one timestep. There are multiple approaches to the modeling of bounces in table tennis research. We implemented a discrete momentum-based model as proposed in [BCW⁺12]. This method provides us with centimeter-level accuracy at the end of the trajectory and is easy to implement.

Following their work, we assume the bounce's effect on the velocity in the $\vec{z}$ direction to be determined solely by a reduction with the coefficient of restitution (COR), a factor that describes the expected energy loss in the process of the bounce.

$$v_{z,t+1} = -v_{z,t} * COR \quad (3.8)$$

The COR can be modeled as a function of v_z , but it changes only minimally, so we decided to use a constant value of 0.9. This value was defined based on the rulebook stating that it should lie between 0.849 and 0.905.

Furthermore, it is assumed that the angular velocity in the $\vec{z}$ direction does not change.

$$\omega_{z,t+1} = \omega_{z,t} \quad (3.9)$$

For the computation of the remaining velocities, we first have to compute the friction coefficient (COF). To do so, we rely on the findings in [ITI⁺17]. They measured the COF for celluloid and plastic balls under a variety of contact point velocities between the ball and the table, finding a linear correlation. We decided to use the correlation they found for plastic balls in our simulation, as that is the more common ball type nowadays.

$$COF = 0.00396 * \|v_t^{cp}\| + 0.2526 \quad (3.10)$$

where v_t^{cp} is the contact point velocity.

Furthermore, we define a denominator D that is used in upcoming computations.

$$D = \sqrt{(v_{x,t} - \omega_y * r)^2 + (v_{y,t} - \omega_x * r)^2} \quad (3.11)$$

where $r = 0.02m$ is the radius of the ball.

Bao [BCW⁺12] differentiates between the two cases by the following equation:

$$COF * (1 + COR) |v_{z,t}| * \frac{1}{D} \geq 0.4 \quad (3.12)$$

If this condition is satisfied, the ball is assumed to be rolling and the remaining values are given by

$$\begin{aligned} v_{x,t+1} &= 0.6 * v_{x,t} + 0.4 * \omega_{y,t+1} * r \\ v_{y,t+1} &= 0.6 * v_{y,t} - 0.4 * \omega_{x,t+1} * r \\ \omega_{x,t+1} &= 0.4 * \omega_{x,t} - 0.6 * v_{y,t+1} / r \\ \omega_{y,t+1} &= 0.4 * \omega_{y,t} + 0.6 * v_{x,t+1} / r \end{aligned} \quad (3.13)$$

Otherwise, when it is sliding, the values are given by

$$\begin{aligned}
v_{x,t+1} &= v_{x,t} - COF * (1 + COR) * \|v_{z,t}\| * (v_{x,t} - \omega_{y,t} * r) * \frac{1}{D} \\
v_{y,t+1} &= v_{y,t} - COF * (1 + COR) * \|v_{z,t}\| * (v_{y,t} + \omega_{x,t} * r) * \frac{1}{D} \\
\omega_{x,t+1} &= \omega_{x,t} - 3 * COF * (1 + COR) * \|v_{z,t}\| * (v_{y,t} - \omega_{x,t} * r) * \frac{1}{D} \\
\omega_{y,t+1} &= \omega_{y,t} - 3 * COF * (1 + COR) * \|v_{z,t}\| * (-v_{x,t} + \omega_{y,t} * r) * \frac{1}{D}
\end{aligned} \tag{3.14}$$

3.3. Test Data Generation

For our tests to be accurate, we need to generate trajectories that are similar to those appearing in real matches. We made several assumptions and restrictions on the ball's initial state and forced some events to happen in the trajectory to reach this goal.

Initial state restrictions All initial state values are randomly sampled from uniform distributions over given bounds. For the initial position, those bounds are set such that typical positions of racket hits are covered. The speed of the ball ranges from 5 to 30 $\frac{m}{s}$. The $\vec{x}$ and $\vec{y}$ velocities are selected such that the trajectory is traveling at a certain angle to the x-axis. The range for this angle is set to be -50° to 50° .

Trajectory restrictions As we do not need to estimate trajectories that violate the rulebook, we can restrict our generated trajectories, trying to represent those rules.

- 1) There must happen a bounce on the enemy side of the table.
- 2) Not more than one bounce is allowed on the own side of the table. Like this, we can still generate serves.
- 3) The trajectory is not allowed to go through a plane representing the net.

Using these restrictions, our trajectories seem to resemble real ones quite well. Two examples of that can be seen in 3.1. You can also find examples of trajectories violating the restrictions in 3.2.

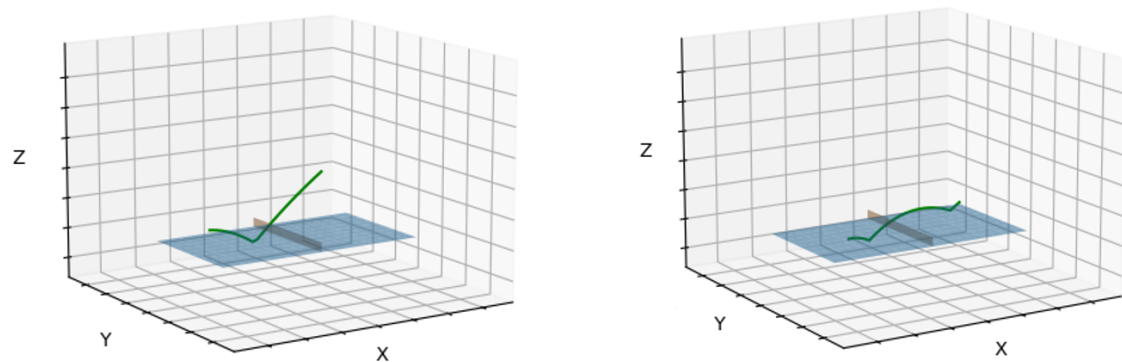


Figure 3.1.: Left: Example for normal trajectory; Right: Example for a serve

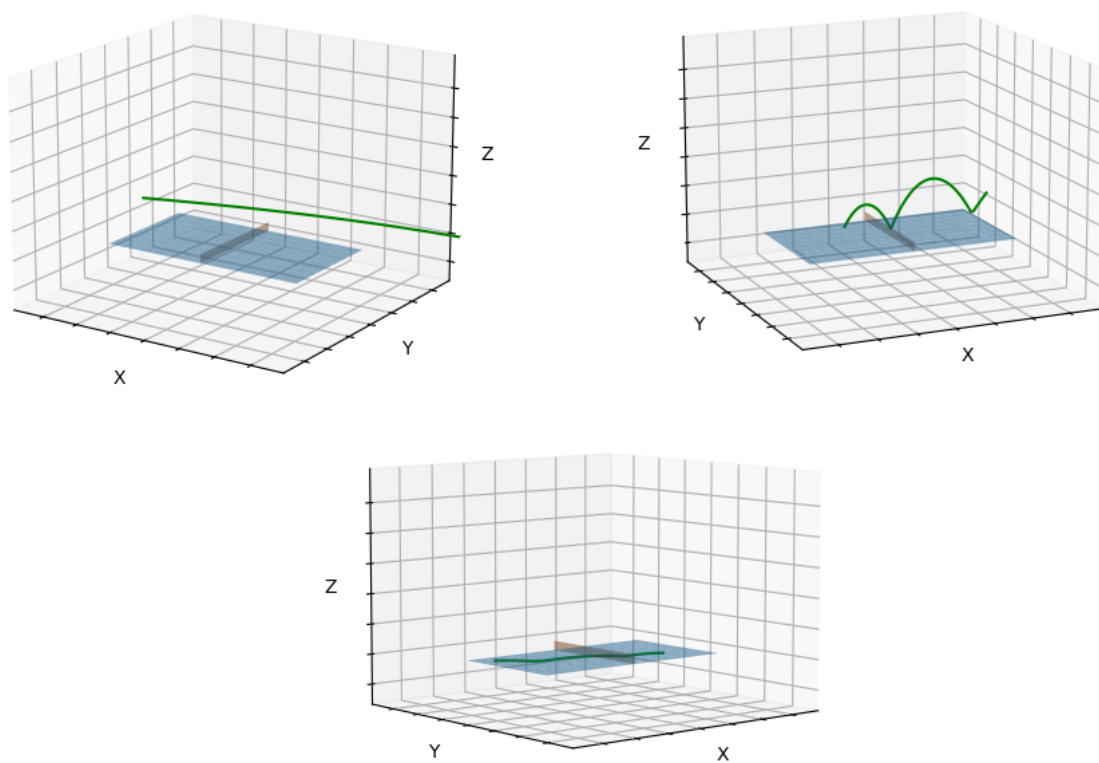


Figure 3.2.: Top Left: No enemy side hit; Top right: Double bounce on own side; Bottom; Trajectory intersecting the net

4. 3D position estimation

When working with only one perspective, we are given two-dimensional data to reproduce underlying three-dimensional data. Such an upscaling of dimensions is only possible with additional knowledge about the environment. That is why we had to estimate the camera's pose first. With that pose, we can generate rays in 3D space that emerge from the camera's position and go in the direction of the given coordinates on the 2D plane.

We can do exactly that with the given 2D ball positions for every frame. We observe strictly separated trajectories that are composed of N recorded frames. Hence, for every frame F_t with $t \in \mathbb{N} \leq N$ we have a given ball position on the camera plane P_t^{cam} . Those positions are used to generate rays R_t that emerge from the camera into the 3D space. The underlying 3D world position P_t of the ball should then lie on these rays.

Making use of the rays, we can reduce our problem to the estimation of a one-dimensional parameter. This parameter is the distance D_i between the camera and the ball's position. A direct measurement of the distance is not possible, as its size on the image can not be perfectly extracted from the discrete pixel space and images can be corrupted by artifacts on the image (e.g. noise and motion blur). Therefore our goal is to find a different way to estimate D_i .

4.1. Using Bounces to estimate trajectory planes

One event that directly gives us an estimation of the distance from the ball to the camera is a bounce. When the ball bounces, it is known to be close to the table. Therefore, when we detect a bounce happening in frame F_t , we can search for an intersection of the ray R_t with the table plane and thus directly receive a 3D estimation of the ball's position.

As the center of the table is the origin of our defined 3D space, the table plane can be defined by the origin and its normal

$$O_T = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, N_T = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \quad (4.1)$$

The rays are defined with knowledge about the camera's translation vector T and rotation matrix M_R , gathered from our pose estimation. It consists of a vector V_R and an origin position O_R which is given by the camera's position in 3D space.

$$O_R = M^T * -T, \quad V_R = O_R + M^T * \begin{pmatrix} P_x^{cam} \\ P_y^{cam} \\ 1 \end{pmatrix} \quad (4.2)$$

where P_x^{cam} is the x position and P_y^{cam} is the y position of the ball on the camera plane. The desired intersection is therefore given by

$$P = O_R + V_R * \frac{O_{R,z}}{V_{R,z}} \quad (4.3)$$

As every match begins with a serve, there are two bounces in the first trajectory. We can use their position estimations P_B^1 and P_B^2 together to create a trajectory plane in space. To define this plane, we make a third point $P_N = P_B^1 + N_T$ by simply adding the table's normal to the first point.

The plane spanned by these three points is now used to estimate the 3D positions of the remaining ball positions in the trajectory. We calculate an intersection with the plane for every ray R_t , directly giving us position estimations P_t for every frame. So the whole trajectory's estimation only consists of spanning planes and finding their intersections with calculated rays. After the serve, any valid trajectory will only contain one bounce. We can not define a trajectory plane from only one position. Hence, we have to find a second position lying on the trajectory plane. The only one that we have an estimation of the position for is at the end of the first trajectory P_{end} , being the point where the first trajectory ends and the second starts. As we already estimated that position, we can use it with the new bounce's position and the table normal to generate the next trajectory plane. This process is further extended for subsequent trajectories. Through this, we receive position estimations for all recorded frames of consecutive trajectories.

Trajectories with spin Since we produce the ball's position estimations with intersections that are all on the same plane, we ignore the possibility of trajectories with spinning balls. Due to the Magnus effect, a spinning ball might result in trajectories with a curvature on the x-y-plane. With our approach, however, we are only able to capture curvature in direction of the table's normal, which resembles the z-axis. This change in direction is even more noticeable when the ball's bounce is affected by spin.

Shallow angles Another problem with this approach is that small errors heavily affect a trajectory's estimation when the angle on the x-y-plane between the camera viewing direction and the trajectory is shallow. The extreme case of this effect would be the angle being 0° , meaning that the camera is placed on the estimated trajectory plane. In this case, any ray generated would have its intersection at the camera's position. A more likely situation is when the camera is very close to the plane which would still result in small errors for the 2D position estimation having an enormous impact on the 3D estimation accuracy.

Discrete data Even when the ball is not spinning and the trajectory is moving orthogonal to the camera viewing direction, another important error-generating factor still remains. As we only have discrete samples given by the frames, the ball's position when a table or racket hit is detected is never in the exact moment of impact. While we would like our algorithm to work with relatively low framerates (e.g. 30 or 60), the distance between an actual bounce position and the one that we were able to detect can become quite large. This can result in a high error for the bounce position estimations. As those give us the trajectory plane to predict all other positions, the whole trajectory is affected by this error.

An error in $\vec{y}$ direction on the image plane causes the generated ray to point further up or down, making the intersection point travel on the table. This effect becomes stronger under shallow angles between the camera viewing direction and the table plane.

It is possible to counteract this problem by estimating the actual impact point of the trajectory, fitting lines or polynomials on the points before and after the actual impact. Their intersection should then be a much more accurate estimation than the discrete points themselves. However, this estimation would still not be perfect and cause errors.

Whole matches When not only looking at single trajectories but at whole matches of table tennis, we can see that the proposed method will probably perform very poorly. Using the last position of an estimated trajectory to form a plane for the subsequent one, we carry its error's effect into future estimations. Progressing through a match, this effect will grow and introduce more randomness in the estimation with every new trajectory. Furthermore, when there is an occlusion in the video, hiding the moment of any racket hit or bounce event, an estimation of that trajectory will not be possible. As this estimation is needed for future trajectories, such a short disruption even makes an estimation for the whole remaining set impossible.

Evaluation In a very restricted environment, this algorithm should be able to reach high accuracy for early trajectories in a match. In our desired use cases, however, there are too many scenarios in which it would fail. Still, it is very efficient as it only relies on simple computations, making it a viable candidate for experimental environments in which real-time application is important. However, a stereo vision system would be the more accurate choice in such a setting, also accounting for curved trajectories.

4.2. Differential Evolution

After evaluating our remaining options, we found that a differential evolution algorithm (DEA) as proposed in [KOA11] is a promising approach to our given problem and used their findings as a basis for our work. They achieved a mean error of only a few centimeters for monocular footage of a bouncing ball and were able to work with short occlusions.

As with any evolutionary algorithm, the DEA keeps a population of N_I individuals I_i with $i \in \mathbb{N} \leq N_I$ being an index ordering them. This population is evolving over a number of generations N_G where $I_{i,g}$ with $g \in \mathbb{N} \leq N_G$ represents the i 'th individual in generation g . Every individual consists of a vector of values $\Theta_{i,g}$ which is initially set to be randomly generated. These values represent the initial state of the ball which allows us to generate their corresponding trajectory in our physics simulation. In each generation, we create new candidate vectors $\tilde{\Theta}_{i,g}$. If those perform better on the given task than their parent with the same index $\Theta_{i,g}$, we substitute them in. An individual's performance is evaluated by a fitness function $F(\Theta)$ which estimates how well the with Θ generated trajectories resemble the underlying truth. Like this, we should receive a population of more accurate estimations with every new generation. Following the guidelines of differential evolution, new candidate individuals are generated under the concept of mutation and crossover functions which are designed to keep characteristics of the parent individual while also exploring the usage of yet untested values.

Fitness function How well an individual performs and hence, if it should replace its parent in the population, is decided by the fitness function. This function is essential for our algorithm, as it controls where the individuals' values move throughout the generations. We build it based on the similarity between the estimated trajectory and the underlying truth. This similarity would be best described by the accumulated error for every position in the trajectory. We further use H to describe hypothetical positions generated by our simulation to resemble the actual positions P of the ball. With $H(\Theta)_t$ being the 3D ball position of a trajectory generated by an individuals values Θ at frame t , this function is given by

$$F^{3D}(\Theta) = \sum_{t=0}^N \|H(\Theta)_t - P_t\| \quad (4.4)$$

The value of this function would be zero if we managed to exactly estimate the underlying true trajectory. As we do not have access to the true 3D positions of the ball for our task, this function can not be applied directly. But since we are able to project 3D points of simulated trajectories to the camera plane, we can measure their similarity on exactly that. So our final fitness function $F^{2D}(\Theta)$ is calculated using the camera plane projections $H^{(cam)}$ which were calculated the same way as in the evaluation of our pose estimation. Given these camera plane positions $pos^{2D}(\Theta)_t$ for an individual at frame t , we are able to measure similarity by computing

$$F^{2D}(\Theta) = \sum_{t=0}^N \|H^{cam}(\Theta)_t - P_t^{cam}\| \quad (4.5)$$

Generation Update Every individual $I_{i,g}$ consists of a vector of parameters $\Theta_{i,g}$, providing an initial ball state. We want to generate new candidate values $\tilde{\Theta}_{i,g}^*$ for each individual by using the concepts of mutation and crossover. In DEAs, new candidate vectors are formed from present individuals and their distances. Those are then used as partners for a crossover with their parents of the same index, creating a mixture of present and new values.

First, we randomly select three individuals out of our current population with indexes r_1 , r_2 , and r_3 which we combine to form a completely new vector $\Theta_{i,g}^*$ by

$$\Theta_{i,g}^* = \Theta_{i,g}^{r_1} + W_g * (\Theta_{i,g}^{r_2} - \Theta_{i,g}^{r_3}) \quad (4.6)$$

with $r_l \sim U(0, |N|)$, $r_{l_1} \neq r_{l_2} \forall l_1, l_2$ and W_g being the differential weight for generation g . W_g controls the amount of mutation, which should become less over the course of generations as we would like our population to explore the value space in early generations and precisely determine the final values in the end stages. As the individuals approach a minimum, their values become less spread, meaning that the amount of mutation will decrease naturally with the value of $(\Theta_{i,g}^{r_2} - \Theta_{i,g}^{r_3})$. Hence, there is no need to decrease W_g . In fact, this effect is large enough that we even have to counteract it by scaling W_g up with another parameter $\delta > 1$ for every new generation.

$$W_{g+1} = W_g * \delta \quad (4.7)$$

Like that, we ensure that the population keeps exploring the value space, making it less likely that it quickly ends up entirely in a local minimum.

We now have crossover partners $\Theta_{i,g}^*$ for each parent individual $\Theta_{i,g}$ in our population. The actual crossover is executed by replacing some of the parent's values with those from $\Theta_{i,g}^*(j)$, where j indicates the position of the value in the vector with dimension J . For every value, if a replacement happens is randomly decided with probability CR . The result of this operation gives us a final candidate vector $\tilde{\Theta}_{i,g}$.

$$\tilde{\Theta}_{i,g}(j) = \begin{cases} \Theta_{i,g}^*(j) & r_j \leq CR \\ \Theta_{i,g}(j) & otherwise \end{cases} \quad (4.8)$$

with $r_j \sim U(0, |N|)$ and $r_k \neq r_l \forall k, l \in J$

As the selection is random, it might happen that no value is changed. This would mean that the individual is not able to evolve in that generation. Therefore, we ensure that there always is some change applied to the parent, by choosing one random index for which the value will always be replaced.

When finally updating our population, we would like to only allow individuals to join the population if they promise a better performance on the task than their parents. For that, we use our fitness function F^{2D} on the parent individual $\tilde{\Theta}_{i,g}$ and its offspring $\tilde{\Theta}_{i,g}$. The values of the new generation's individuals are thereby determined with

$$\Theta_{i,g+1} = \begin{cases} \tilde{\Theta}_{i,g} & F^{2D}(\tilde{\Theta}_{i,j}) \leq F^{2D}(\Theta_{i,j}) \\ \Theta_{i,g} & F^{2D}(\tilde{\Theta}_{i,j}) > F^{2D}(\Theta_{i,j}) \end{cases} \quad (4.9)$$

Like this, the population should converge to a local minimum on the fitness function. Our goal now is to make it likely that this local minimum also represents the global optimal solution, as that should represent the real trajectory's initial state.

One way of encountering this problem would be to optimize the used hyperparameters for our model. Generally, choosing larger numbers of generations N_G and individuals N_I results in the initial population covering the value space more densely. This makes it more likely that some individuals' values end up being close to the global optimum. However, choosing large numbers for those parameters also results in a longer computation time. Despite us not being interested in making our algorithm capable to run in real-time, we would like it to be usable in a reasonable time window. Hence, we had to compromise on performance, choosing our parameters to be $N_G = 300$ and $N_I = 100$.

For the remaining parameters, it is not as obvious how to choose them to generate the best possible results. Selecting larger values for W_0 , δ , and C_R means that the DEA will explore more of the value space, but also reduces the fine-tuning of values. So if we find that our algorithm often ends in a local minimum, we should increase them. Otherwise, if it often ends up with values close to the global optimum, but is unable to reach close enough to its exact values, we would like to lower our parameters. As we can not analytically solve this problem, we have to benchmark our algorithm with different values to find the best fit. With our limited time for this project in mind and taking into account the quite long computation times, we decided to skip this process and select the values used in [KOA11]. Since they deal with a very similar problem and use the same fitness function, we decided that this is a reasonable compromise.

4.2.1. Choosing values for Θ

The state of a ball is given by the vectors for position p_t , velocity v_t , and angular velocity ω_t with $p, v, \omega \in \mathbb{R}^3$. To generate trajectories with the help of our simulation we need the initial state of the ball given at $t = 0$. Our evolutionary algorithm is designed to estimate exactly those initial states. Hence, it seems prudent to include every value from those three vectors in the individuals' values Θ , making our algorithm solve a problem in $\mathbb{R}^9$.

However, problems of lower dimensionality are generally easier to solve, which made us think about how to reduce the number of values in Θ . When looking back at our first approach that estimates the trajectory plane, we realized that we did exactly that for the position by generating rays R_t in 3D space for given 2D positions P_t^{cam} on the camera plane.

If we apply this technique to the first position in a trajectory, we can make our evolutionary algorithm optimize only for the distance D to the camera to find the position. This results in a $\Theta = \{D, v_{0,x}, v_{0,y}, v_{0,z}, \omega_{0,x}, \omega_{0,y}, \omega_{0,z}\}$ with a dimensionality of seven.

When we want to simulate our values, we can reproduce a usable initial position with

$$p_0 = O_R + D * R_0 \quad (4.10)$$

Integrating this dimension reduction step greatly improved the performance of our algorithm.

We also tried to apply a similar technique to reduce the dimensionality of the velocity. This was done using the plane spanned by the two rays from the first and second positions of a trajectory. With two distance values D_v^0 and D_v^1 in Θ , we were to reproduce a velocity by

$$v_0 = O_R + D_v^0 * R_0 + D_{vel}^0 * R_1 \quad (4.11)$$

Unfortunately, this did not improve the performance of our estimation. That probably is the case because of the Magnus force and gravity changing the trajectory's velocity direction. This means that while we calculate our plane based on a linear observation, the trajectory is actually curved between the two points we used to do so. The estimation of the trajectory plane might reach good enough accuracy to improve our algorithm if the framerate was higher, but as we would like to work with relatively low framerates, we quickly abandoned this idea. Nonetheless, looking at the accuracy increase we gained from reducing the dimension used to produce initial positions, we think that it could be a valuable addition to other projects.

4.2.2. Restricting validity of individuals

When testing our algorithm, we noticed that the final result often ended up being a trajectory that matched the 2D positions well, but actually consisted of completely different 3D positions. When further analyzing this problem, we realized that in most of these situations, the trajectory does not resemble a valid move in table tennis. That is why we decided to bias the individuals by directly overwriting their fitness values.

We first need to differentiate between valid and invalid trajectories. For this purpose, we made three conditions that have to be true for a trajectory to be valid.

- a) The trajectory does not travel below the table. If it does, we can be certain that it will not reach the surface of the table, making it an invalid move.
- b) The trajectory bounces off the enemy side of the table. This event occurs in any valid move in table tennis.
- c) It starts at a reasonable position. Even if we set reasonable bounds for our initial state's values, the DEA can evolve into states that break those. As we know for certain that these bounds should hold, we mark such trajectories as invalid.

We then proceeded to punish individuals that violate these rules by fixing their fitness function output to be a constant $C_F = 10000$. This constant was designed such that valid trajectories are guaranteed to yield lower values than invalid ones. Like this, we rule out that invalid trajectories infest our population by not allowing them to overwrite existing ones, although the actual fitness function might have produced a lower value.

While this stopped the convergence of our population to trajectories with high error and low fitness function values, it also created scenarios where the population did not make any progress. That is because little to no generated population members were valid, meaning that they all have a fitness value of C_F , resulting in no clear direction for the algorithm to evolve to.

To counteract this, we kept C_F as a baseline to handle invalidity, but also made additional punishment functions to guide the population in a reasonable direction. Every above-mentioned case to determine if a trajectory is valid received its own punishment variable.

- a) If the trajectory is below the table, we introduce punishment variable a which gives the distance between the table plane and the lowest position of the trajectory.

$$a = H_t - \min_t(p_{t,z}) \quad (4.12)$$

where H_t is the height of the table. Therefore, trajectories closer to the table's surface receive lower values.

- b) If the ball does not hit the enemy side of the table, we introduce variable b , given by the distance from the trajectories ending position to the center of the table on the x-y-plane

$$b = \left\| \begin{pmatrix} p_{N,x} \\ p_{N,y} \end{pmatrix} \right\| \quad (4.13)$$

This should bring the end of trajectories closer to the enemy's side of the table.

- c) If the start position is out of our defined bounds, we introduce c , being the accumulated axe values on every bound that was broken. For the calculation, we introduce the values B_x , B_y , and B_z , which are 1 if the indexed axis value breaks the bounds and 0 otherwise.

$$c = (|p_{0,x}| \quad |p_{0,y}| \quad |p_{0,z}|) * \begin{pmatrix} B_x \\ B_y \\ B_z \end{pmatrix} \quad (4.14)$$

Here, lower values for c are achieved when the trajectories initial position comes closer to our bounded area.

These functions are only activated when the trajectory violates the specific rule. For rules that are not being violated, we simply make their punishment variable value 0 to not bias them further in that regard. Finally, we end up with a fitness function for cases in which the trajectory is invalid F^{ival} given by

$$F^{ival}(I) = C_F + a + b + c \quad (4.15)$$

This fitness function is by no means a good indicator of if a trajectory performs well, but it leads to the population consisting of more valid trajectories. Once that happens, the real fitness function will overtake for the task of actually optimizing for our given 2D positions.

After implementing this extra punishment, we did not observe any more cases where the population was unable to learn. In fact, our performance drastically improved after making these changes.

4.2.3. Determining start and end frames of trajectories

As we made various restrictions on the validity of trajectories generated from our population, we heavily rely on being able to detect the start and end points of trajectories in a set. As every trajectory begins with a racket hit, our goal is to determine those to directly infer start and end frames. We tried to do so by observing the change in velocity of the ball on the camera plane but failed to detect racket hits reliably under all observed perspectives. That is because bounces also resemble such an instant change of velocity and therefore might trigger the racket hit detection.

We decided to ignore this problem for our research and focused on improving the DEA with given trajectory frame bounds. With additional information about the racket's movement, however, this task becomes much simpler. Then, a change of velocity can be used as a first indicator for a hit while its position relative to the rackets can reassure if it should be counted as one. [VFB20] introduced a method to track racket's positions and spot events in which they are likely to be attacking the ball. Using such a method, racket hit detection will become very reliable and our DEA will become a viable option even for the full duration of matches.

4.2.4. Dealing with spin

Including spin in the values estimated by the evolutionary algorithm did not improve the accuracy. Due to the limited time given for this project, we were unable to analyze this problem further and possibly find a solution to it. Therefore, we decided to remove spin from the individuals' value vectors, making the initial position and the velocity the only parameters to estimate. Thereby, we indirectly make the assumption that the ball does not spin when making our estimations. This reduces Θ to D, v_x, v_y, v_z , lowering its dimension to 4 and hence making it an easier problem to solve.

Nonetheless, we believe that, with more time available, it would be possible to further increase the estimation accuracy by including angular velocity in our algorithm. In fact, [KOA11] included spin in their project and managed to estimate it well with their algorithm. One reason why it did not work in our case might be that we observe much fewer bounces in our trajectories than they did in their work. As spin changes the trajectory much more drastically when a ball collides with the table, observing many bounces should make estimation much easier.

4.3. Method comparison

Looking at the two proposed approaches, there is one critical point in which they differ. The first approach tries to find a solely analytical solution to the problem, while the second one combines analytical properties and restrictions with empirical techniques.

Theoretical limits For the theoretical limits of both approaches, we assume perfect data for given 2D ball positions and the camera pose. Furthermore, we also assume a perfect physics simulation to be used for the DEA.

Under such circumstances, it would still be impossible to find the perfect solution by applying the first method. This is because it ignores the possibility of spin in its assumptions trying to fit a three-dimensional curve into a two-dimensional one. But even if we assume no spin, it would be limited by a discrete number of frames. Bounce positions are never perfectly given by those, making it impossible to find the exact trajectory plane.

When looking at the evolutionary algorithm, however, there is only one possible underlying true initial state of the ball, best representing the ground truth. An individual with values according to that state would achieve zero error for the fitness function. Hence, it resembles a global minimum which can theoretically be found and estimated arbitrarily well by the evolutionary algorithm. But for that, the angular velocity would have to be present in Θ .

Real world application The analytical approach struggles a lot with shallow perspectives and error, as it relies only on only two positions to estimate the whole trajectory. When looking at the evolutionary algorithm, it is much more stable under such circumstances by using the whole set of points to generate an estimation.

More importantly, the evolutionary algorithm calculates estimations for each trajectory independently, making it a viable option for the estimation of whole matches of table tennis. This probably is its biggest advantage over the analytical approach which had to make use of previous estimations, unavoidably generating more random estimations with every new trajectory. With the DEA, even when input points are missing due to occlusion, we can still give an estimation of their 3D position as we are able to sample our simulated trajectory at any

point. Expanding this thought, we can also give estimations of positions at any other point in time by an adjustment of dt .

Probably being the only important drawback of the DEA, it is much slower than the analytical solution. This stems from the fact that we have to calculate multiple forces for every step in our simulation. Furthermore, this has to be done for not only one trajectory, but for a whole population of them over many generations. Meanwhile, the analytical approach only has to calculate intersection points once for every frame.

5. Evaluation

To show the performance of our DEA, we generated a huge amount of test trajectories with our physics simulation, aiming to carry out a Monte Carlo test. We tried to make those trajectories as realistic as we could, accounting for a wide range of start positions, velocities, and angular velocities. While we ignored the ball's spin in our algorithm, we still wanted to test how well it performs on trajectories that include it, as that gives us a better idea of how it will perform in real-life scenarios and possibly lets us observe interesting differences in their estimation accuracy.

That is why we split our benchmark into two separate phases. First, we analyze the performance for single trajectories without spin. Those can be either serves or normal ones with a single bounce on the enemy's side. We generated 500 of them, all starting from the same side of the table. We then mirrored them such that we end up with a total of 1000 trajectories from an equal distribution of starting side. The same was done for trajectories with spin to compare the results under equal conditions. The rotation axis was chosen randomly and the rotations per minute were sampled from a uniform distribution ranging from 0 to 9000.

Since we need two-dimensional positions as input for our algorithm, we used known camera poses extracted from videos and projected our simulated 3D points onto their corresponding camera planes. For this, we selected three common perspectives in table tennis videos 5.1. While that makes our testings immune to the error of the pose estimation, we should still be able to find correlations and significant observations in the results. The chosen perspectives cover the full range of angles, maximizing the amount of variety we can achieve. The videos we used all stem from the copyright-free Youtube channel of Tennis2table.com. Those were also used to generate the figures used in this work.

Because we need 2D positions as input for our algorithm, we use the gathered poses from the given perspectives to project simulated 3D trajectories on the image plane. By that, we receive perfect 2D positions on the image plane. In a real scenario, however, we would have to deal with errors in their estimations. That is why we did not only test our algorithm's performance with perfect projections but also introduced some errors in their positions. This was executed by applying Gaussian noise with mean 0 and standard deviation values of 0, 1, and 2 cm to both, the x- and y-coordinates on the image plane. The algorithm was tested separately on the three created scenarios with different noise amounts, allowing us to directly compare their performance. Sampling was done 60 times per second, representing a reasonable framerate that is present in many videos on the internet.

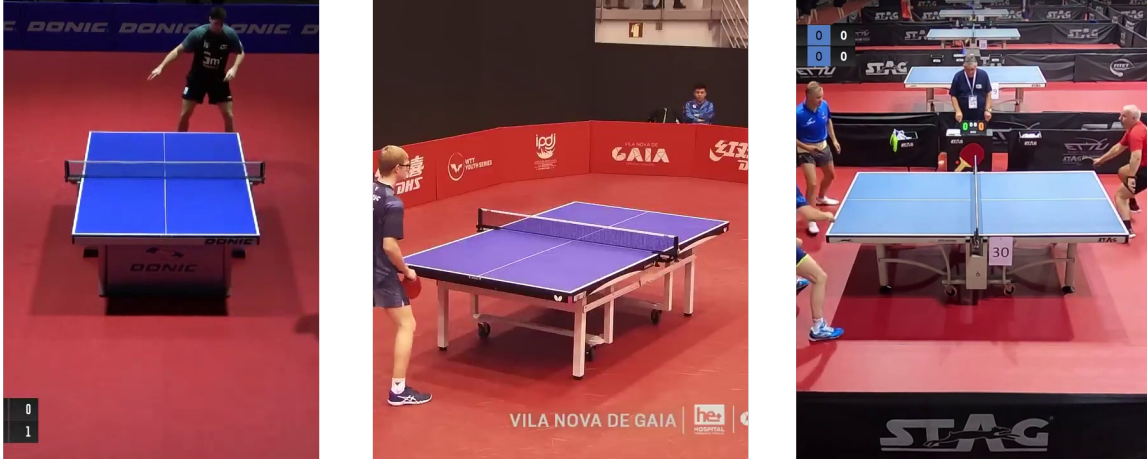


Figure 5.1.: The three perspectives used for our benchmark.

5.1. Analysing Correlations

Bounces effect on accuracy When analyzing our data, we found a strong correlation between the point in time where the ball bounces on the table and the accuracy of the position estimations. When looking at figure 5.2, we can see that for both, the case without and with spin, the estimation error falls with ongoing time until it reaches a global minimum. After that, it starts to rise again. We explain this by bounce events typically happening at that time in the trajectory's evolution. The long downfall of error is caused by the ball first having to travel across the whole length of the table until it bounces off it. Shortly after, the trajectory ends and at that point, a racket hit would occur. To emphasize this relation, we made figure 5.3. There, we can observe a linear relationship between the accuracy of an estimated position and its time distance to the closest bounce event for the case of no spin. Looking at the spin curve, we can observe a similar increase at first, but going further in time leads to the error falling again. We are yet unsure what causes this effect and would have to analyze this property further to make assumptions about its origin.

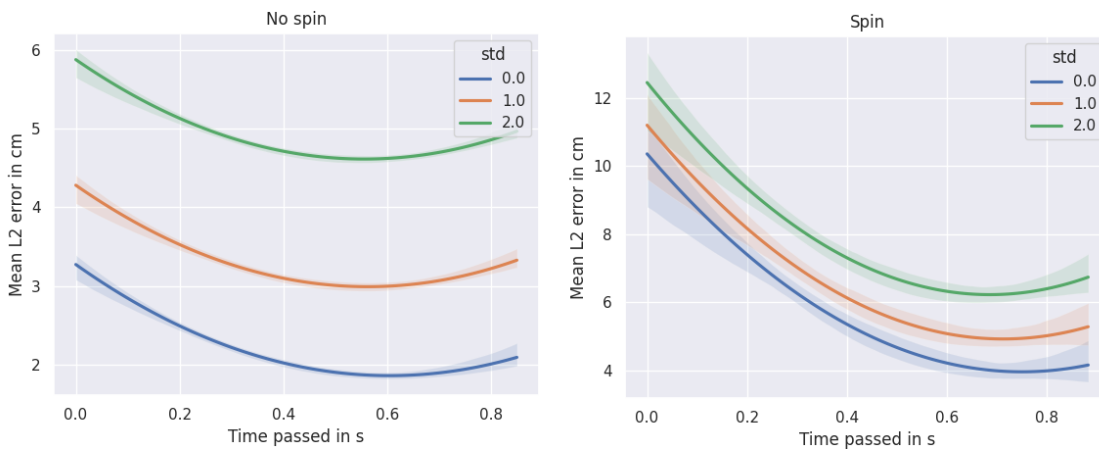


Figure 5.2.: The mean error evolution for all tested trajectories over the travel time since the start point. The plot was created by fitting a polynomial of degree 2 on the recorded data.

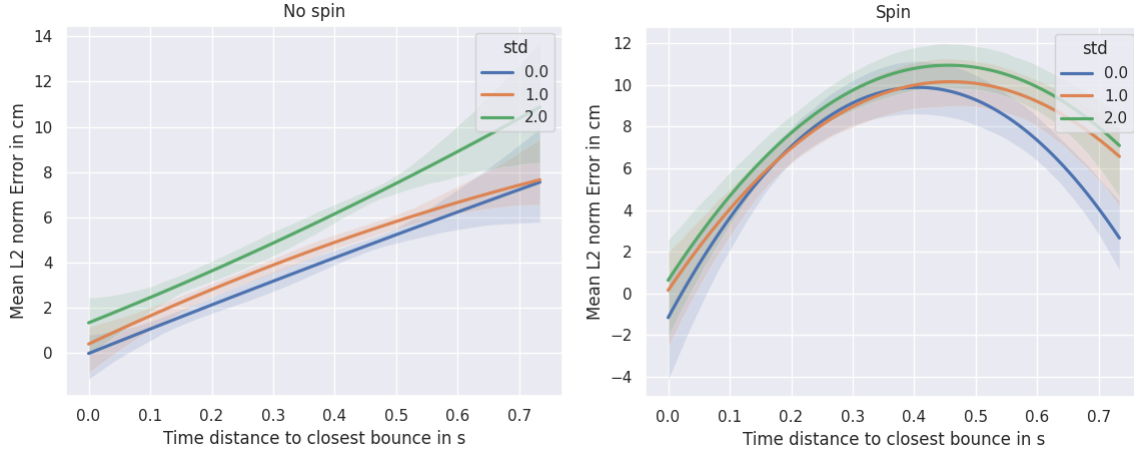


Figure 5.3.: The mean error evolution over the time distance to the closest bounce event. The plot was created by fitting a polynomial of degree 2 on the recorded data.

Still, for both cases, the error is by far at its lowest for frames close to bounce events. This emphasizes the importance of those events for our estimation. Looking back at our first analytical approach to this topic, we there solely used bounce events for all estimations. That was because they yielded the best point candidates to estimate the distance with. With our DEA, a similar dependence seems to be present.

Initial position error effect To understand better where the in 5.3 found linear correlation stems from, we used our visualization tool to have a look at trajectories that were poorly estimated. Doing so, we observed that those trajectories look almost perfectly fitting from the camera's perspective, as can be seen in figure 5.4. This means that the DEA did indeed find a good minimum for the fitness function, but failed to estimate the 3D positions. We further analyzed this problem, finding that the bounce point usually makes the crossing point of the estimated and the true trajectory. This observation together with 5.3 proves the bounce dependency assumption we made for the first analyzed graph 5.2.

To better understand why the relation between bounce distance and error is linear, we made figure 5.5. It provides a simplified two-dimensional view of the table comparing the two trajectories. Here, we directly see the error of the estimation linearly scaling with the distance from the bounce position. This figure only makes sense when we think about trajectories without spin. Adding spin, it can not be simplified like that, which also explains why we did not observe a linear relation for that case in figure 5.3.

Continuing this thought, we would expect our initial position error to be a direct indicator of the estimation performance of the whole trajectory. In fact, when comparing those, we found a linear correlation with a coefficient of about 0.5 for the case of no spin and 0.45 for trajectories with spin. Figure 5.6 shows that correlation.

So if we want to further improve the performance of our DEA, our main focus would be to improve the accuracy of the initial position estimation. The remaining position errors should then naturally decrease as well. Unfortunately, this is a difficult task, as the fitness function seems to yield almost perfect values for trajectories far away from the underlying truth. Theoretically, the actual trajectory should reach even lower values on the fitness function. But with many low local minima, it becomes too difficult for the algorithm to find the global minimum.

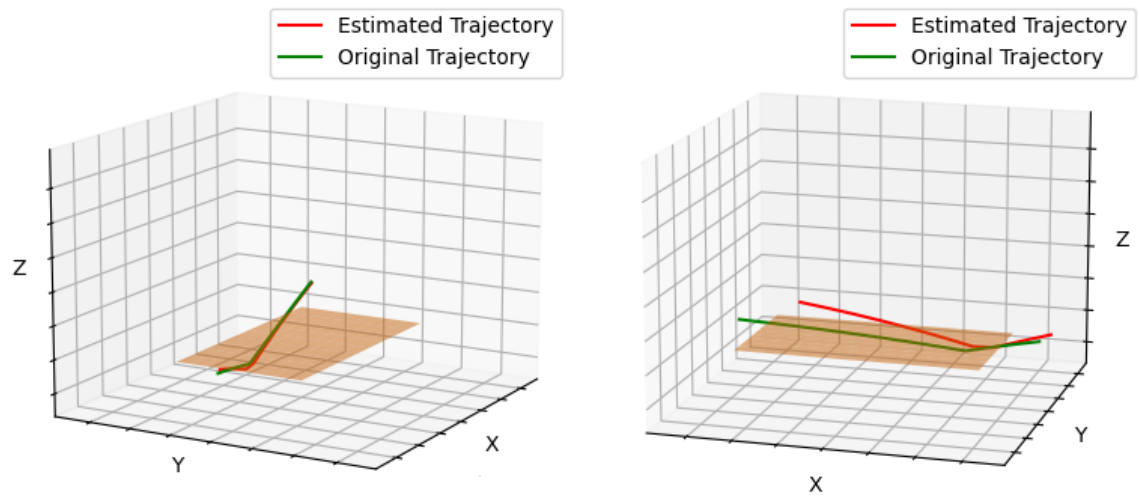


Figure 5.4.: The same trajectories from different viewing angles. The left image shows the camera's viewing angle.

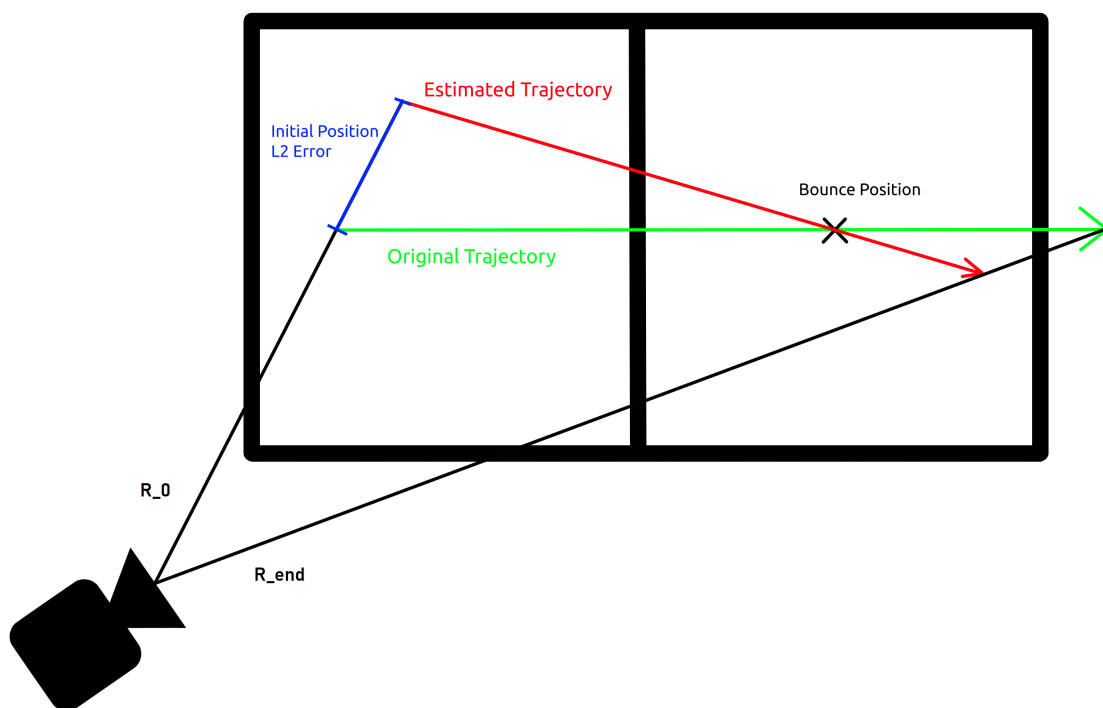


Figure 5.5.: Simplified 2D figure of two trajectories without spin to show the linear correlation.

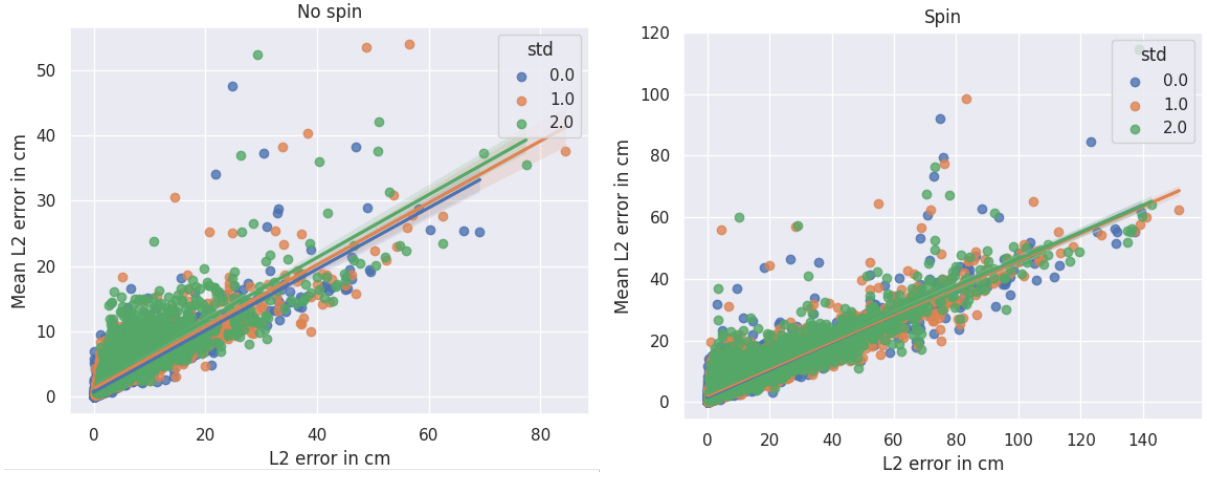


Figure 5.6.: Mean trajectories errors over their error for the first position. We fitted a line on the data to show a linear correlation.

Perspective influence on accuracy To analyze how well our DEA handles different viewing angles, we not only used multiple perspectives for our tests but also observed the change of error over the angle between the trajectories' initial direction and the camera's viewing direction on the xy-plane. Our findings are shown in figure 5.7. For the case of no spin, we observe a clear decrease in error as the camera comes closer to being orthogonal to trajectory. We explain this by the sampled 2D camera plane points being closer together, making it more difficult to comprehend the distance of the ball to the camera. This effect is again similar to what we noticed about the first introduced analytical algorithm. But, in contrast to that, here it does not seem to worsen dramatically with errors in our camera plane positions. Even when a Gaussian error with a standard deviation of 2 is applied to the positions, we reach a reasonable mean error of about 5.5cm for extremely shallow angles.

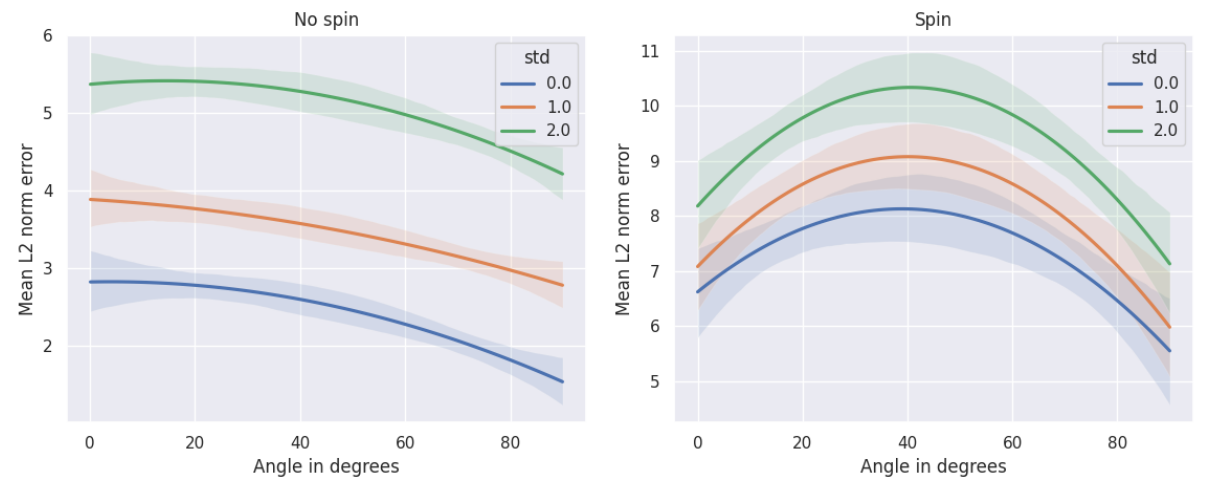


Figure 5.7.: Mean error for trajectories over the angle between their initial movement direction and the camera's viewing direction. The plot was created by fitting a polynomial of degree 2 on the recorded data.

For trajectories with spin, however, we make a different observation. There, the error is low for shallow and large angles but rises for values of about 45 degrees. We explain that by a strong effect of spin on the estimation if they force the ball to curve in direction of the camera or away from it. As estimating the distance of the camera to the ball is the most difficult part of our task, adding more variation in that direction should make our DEA perform worse. We assume that this effect is strongest at an initial trajectory angle of about 45°.

Velocities influence on accuracy When analyzing the change of error over the speed at which the ball travels, we did not find a strong correlation. As can be seen in figure 5.8, the error does not change much for our tested speed interval. This proves that a framerate of 60 is enough to sample balls with a reasonable speed for table tennis. These findings also lead to the supposition that increasing the framerate will not yield any further benefits to accuracy. This once again shows the stability of the DEA compared to the analytical approach which has a performance directly dependent on the framerate.

When looking at trajectories with spin, however, we observe less error with higher velocities. We assume that this is caused by slower trajectories generally having more airtime, allowing for the Magnus effect to change their direction more in the observed timeframe.

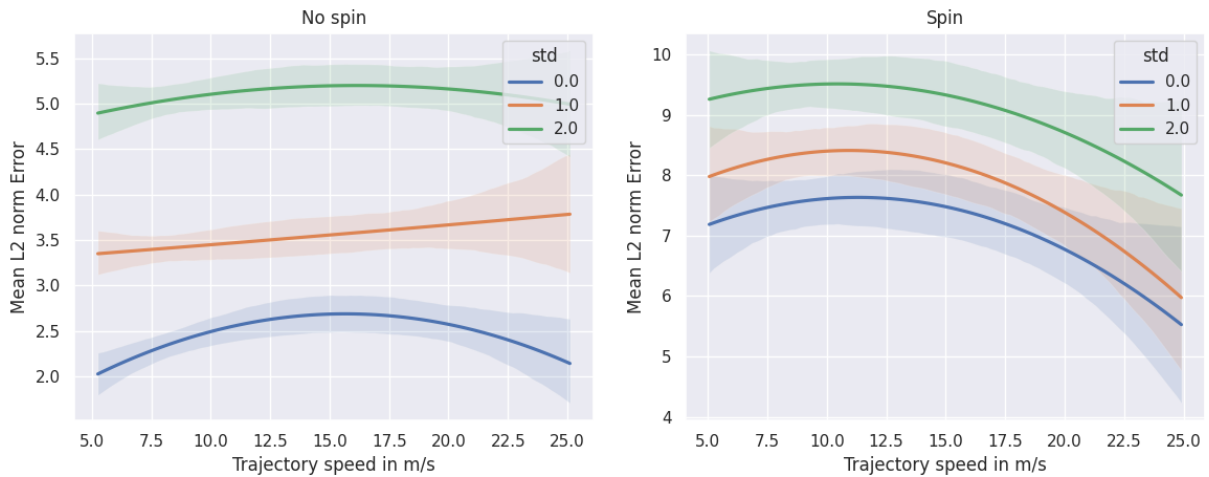


Figure 5.8.: Mean trajectory error over the initial speed. The plot was created by fitting a polynomial of degree 2 on the recorded data.

5.2. General performance

Our algorithm seems to perform well under perfect conditions with a mean error of 2.45cm. But even if we add noise with a standard deviation of 2 and add spin to the trajectories, it still manages to reach an acceptable mean error of 9.25cm. Since we did not include spin for trajectories generated by our DEA, we expected a higher error on this task.

std	Mean	Median
0	2.45	1.36
1	3.49	2.54
2	5.09	4.16

Table 5.1.: Comparing mean and median error on trajectories with no spin

std	Mean	Median
0	7.31	3.57
1	8.05	4.43
2	9.25	6.11

Table 5.2.: Comparing mean and median error on trajectories with no spin

We would also like to point out that this mean is heavily influenced by outliers in our tests which can be seen in figure 5.9. That plot also shows that the median error is much lower than the mean we received. We would have to further analyze our data to understand where these outliers come from, but if we can find their cause and counteract it, the median could be a good indicator of the average accuracy we can expect to be possible.

The influence of outliers becomes even clearer when looking at figure 5.10. There we can see that even under influence of Gaussian error with a standard deviation of 2, almost 80% of the trajectories were estimated with an average error of 7.5 cm or less and almost all trajectories reached a mean error of less than 20cm. Although the average error of trajectories with spin is a lot higher, they too achieved less than 20cm for almost 90% of trajectories. For convenience, we also made two tables comparing the mean and median values in all test cases (5.1).

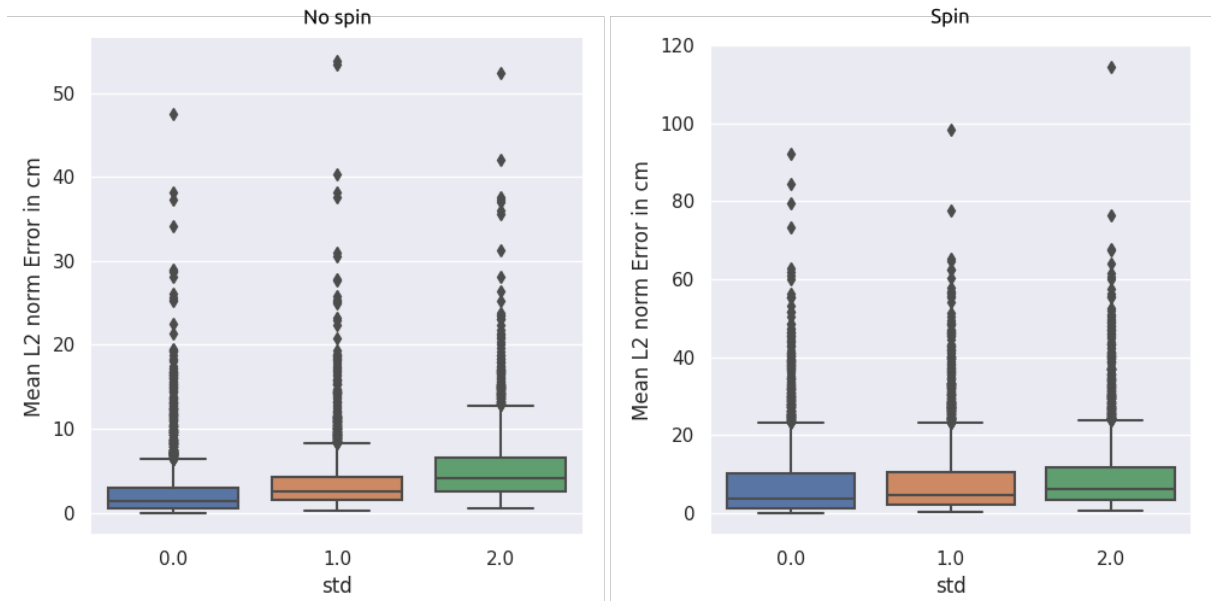


Figure 5.9.: Boxplots showing how the mean L2 norm error of trajectories is spread.

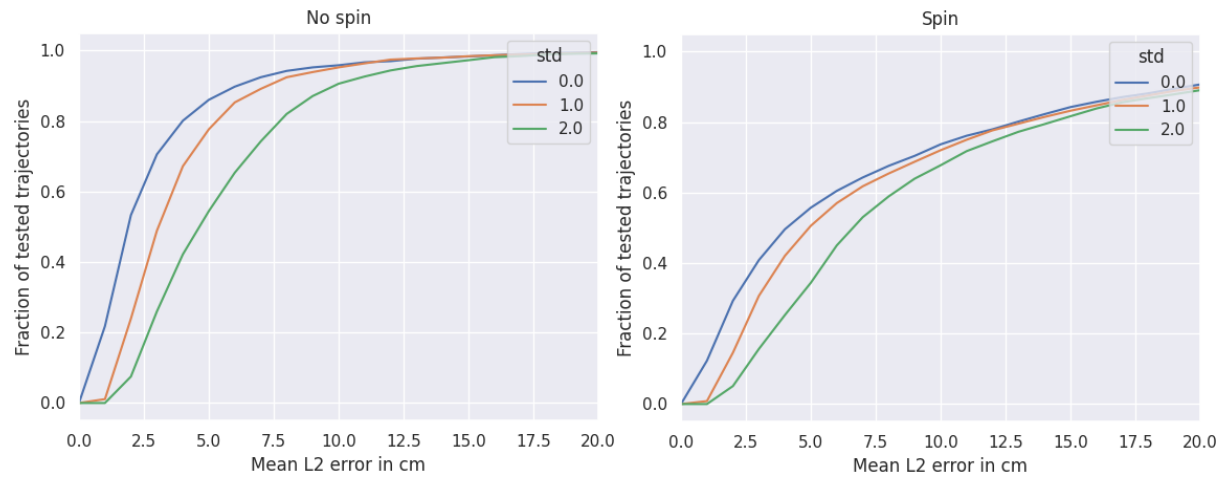


Figure 5.10.: Fraction of tested trajectories that managed to achieve a certain error that is displayed on the x-axis.

5.3. Tests with real data

As much as we tried to resemble real data with our simulated trajectories, applying it to real data is necessary to prove our concept. Hence, we also tested 16 trajectories in an experimental setting with recorded 2D and 3D positions of the ball. The two used perspectives can be seen in fig 5.11. As the table is black in this scenario and does not stand out much from the background, we could not rely on our corner estimation and had to manually set the positions of the corners. But we expect that our algorithm would have performed similarly well if the table had a usable color since there is no occlusion present in the video.

We observed an average error of 15.4 cm for the first perspective in 5.11 and 8.3 cm for the second. The first perspective's camera viewing direction was very close to the tested trajectories movement direction, explaining why the error is so much higher. We expect even less error than 8.3 cm for perspectives recording the trajectories with an angle closer to 90 degrees.

Although worse than our test results with no spin, these values clearly show that a DEA is able to estimate the 3D positions in a real-world scenario with a manageable error. However, it is important to keep in mind that the prediction of the first position is usually by far the worst. As we observed a linear correlation with factor 0.5 in 5.6, the first error should usually be about twice as high as the mean we observe.



Figure 5.11.: The two perspectives we used for our tests

5.4. Concluding Thoughts

With our work, we showed that a 3D estimation of trajectories only from 2D image data of one perspective is possible. Furthermore, using a DEA algorithm, we should achieve manageable errors on casually filmed videos, coming close to our goal of generating data to train a machine learning algorithm.

Nonetheless, accuracy varies a lot along the trajectory, being worst at the start and best at the point of bounce events. This is a significant problem unless we are only interested in well-estimated ends of trajectories. We assume that the first position error can be reduced further, but that will be a tough challenge, as there is no direct indicator for distance at that point. We think that achieving this goal would benefit the performance much more than the inclusion of spin could, as that only influences the trajectory a small bit, while initial position errors of more than 20cm are a common occurrence.

If we tried to expand our work and create usable data with it, we would limit ourselves to videos shot from the table's side, as can be seen in the third perspective of figure 5.1. Like this, the average angle between the camera's viewing direction and trajectory should be at its lowest. As this perspective is very common in table tennis, there should still be enough data available to generate a large dataset.

Finally, we conclude that DEA has a large potential for the presented problem and believe that it can become a useful tool to generate data. But for that, a further reduction of initial position error is probably necessary. To achieve that, we would have to either enable our algorithm to be better at finding the global minimum or include an indicator of distance for the first position. Such an indicator might be possible when also using data on the racket's movement.

A. Value Table

Table A.1.: Value Table

Short	Long	Value	Additional note
$C_{green} >$	Lower bound for green	[90, 30, 90]	HSV Colorspace
$C_{green} <$	Upper bound for green	[135, 255, 255]	HSV Colorspace
$C_{blue} >$	Lower bound for blue	[40, 30, 90]	HSV Colorspace
$C_{blue} <$	Upper bound for blue	[75, 255, 255]	HSV Colorspace
C_{dist}	Distortion coefficients	[0, 0, 0, 0]	
M_{cam}	Camera matrix	$\begin{pmatrix} W & 0 & W/2 \\ 0 & H & H/2 \\ 0 & 0 & 1 \end{pmatrix}$	
t_{sim}	Line similarity threshold	$W/20$	
t_D	PnP validity error threshold	$H/15$	
t_{p_dist}	Perspective similarity point distance threshold	$H/40$	
C_D	Drag coefficient	0.4	
C_M	Magnus coefficient	0.069	
COR	Coefficient of restitution	0.9	
C_F	Invalid trajectory fitness constant	10000	
dt	Simulation step size		

B. Abbreviations

DEA	Differential Evolution Algorithm
RL	Reinforcement Learning
M_R	Rotation Matrix
S_{acc}	Set of estimated poses labeled as accurate
dt	Simulation step size

Bibliography

- [BCW⁺12] Bao, Han, Xiaopeng Chen, Zhan Tao Wang, Min Pan Fei Meng: *Bouncing model for the table tennis trajectory prediction and the strategy of hitting the ball. 2012 IEEE International Conference on Mechatronics and Automation*, 2002–2006. IEEE, 2012.
- [DH72] Duda, Richard O Peter E Hart: *Use of the Hough transformation to detect lines and curves in pictures*. *Communications of the ACM*, 15(1):11–15, 1972.
- [ITI⁺17] Inaba, Yuki, Sho Tamaki, Haruhiko Ikebukuro, Koshi Yamada, Hiroki Ozaki Kazuto Yoshida: *Effect of changing table tennis ball material from celluloid to plastic on the post-collision ball trajectory*. *Journal of human kinetics*, 55(1):29–38, 2017.
- [KOA11] Kyriazis, Nikolaos, Iason Oikonomidis Antonis Argyros: *Binding vision to physics based simulation: The case study of a bouncing ball. Proc. BMVC*, 2011.
- [NNH10] Nonomura, Junko, Akira Nakashima Yoshikazu Hayakawa: *Analysis of effects of rebounds and aerodynamics for trajectory of table tennis ball. Proceedings of SICE Annual Conference 2010*, 1567–1572. IEEE, 2010.
- [VFB20] Voeikov, Roman, Nikolay Falaleev Ruslan Baikulov: *TTNet: Real-time temporal and spatial video analysis of table tennis. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 884–885, 2020.